Emma's Librarian: Uma ferramenta de apoio à pesquisa integrando inteligência artificial e buscas bibliográficas unificadas

João P. V. Gonçalves[1], **Aline de Lucas**[1], **Ana P. A. C. Shiguemori**[1]

[1]Instituto Federal de Educação, Ciência e Tecnologia de São Paulo (IFSP)
CEP 12322-030 – Jacareí – SP – Brasil

joao.verissimo@aluno.ifsp.edu.br, {delucas,anapaula.acs}@ifsp.edu.br

Abstract. *Developing a research project involves a variety of tasks—ranging from meetings, activity logging, and task management to the analysis of academic literature and the preparation of conference papers and journal articles, among many other activities—which are typically carried out using a multitude of tools. Against this backdrop, this article describes the development methodology and results of Emma's Librarian, a tool designed to meet various researcher needs through a single, local, and private platform.*

Resumo. *O desenvolvimento de um projeto de pesquisa envolve uma variedade de tarefas, desde reuniões, registro do que foi realizado, gestão das tarefas, análise de trabalhos acadêmicos, publicações em congressos e revistas, dentre outras atividades que frequentemente são realizadas com a utilização e o suporte de uma pluralidade de ferramentas. Nesse contexto, este artigo descreve a metodologia de desenvolvimento e os resultados obtidos pela plataforma de Emma's Librarian, uma ferramenta que tem como objetivo atender a diversas demandas dos pesquisadores com um único espaço local e privado.*

1. Introdução

O processo de pesquisa científica envolve, entre outras atividades, a busca de referências que justifiquem sua pesquisa, sustentem a argumentação e as afirmações realizadas durante a redação do artigo, bem como fundamentam a hipótese central. Esses dados, por sua vez, podem vir de diversas fontes confiáveis, incluindo principalmente repositórios voltados para uso acadêmico, como Scopus, Web of Science, OpenAlex e Crossref. Tais repositórios apresentam diferentes formas de buscar trabalhos acadêmicos, diferentes interfaces e sintaxes para busca estruturada [Bramer et al. 2018], bem como diferentes plataformas para realizar essas buscas.

Revisões da literatura, em sua pluralidade de tipos, podem exigir a gestão de grandes quantidades de artigos, arquivamento de materiais não alinhados com o objetivo da pesquisa encontrados pelos critérios de exclusão e inclusão, dentre outras divisões. Eventualmente, algumas revisões podem preferir categorizar os trabalhos coletados a partir de critérios escolhidos pela pesquisa, viabilizando uma exibição posterior dessa categorização na forma de tabelas ou gráficos.

Essa fragmentação de buscas em diferentes plataformas, somada a outros meios de organização do projeto, como diários de registro do histórico, *links* de reuniões e quadro de tarefas, gestão de artigos e documentos, inspirou a criação de uma ferramenta

que facilitasse alguns desses processos, centralizando em um único espaço seu gerenciamento. Este artigo apresenta o desenvolvimento, funcionalidades e resultados do software Emma's Librarian, abordando a metodologia utilizada no seu desenvolvimento, resultados obtidos e métodos de validação.

2. Metodologia

Com a evolução dos *harness* de agentes de IA [Yang et al. 2024], o desenvolvimento de software com código majoritário ou inteiramente escrito por inteligência artificial tornou-se possível quando combinado com técnicas de engenharia de software amadurecidas durante décadas na história [Beck 2000]. O desenvolvimento orientado a testes (TDD) [Beck 2002], tornou-se um pilar de segurança para o desenvolvimento de software assistido por agentes de inteligência artificial [Akita 2026]. Essa prática garante que eventuais regressões sejam detectadas por uma extensa suíte de testes automatizados (atualmente composta por 772 testes), permitindo a correção prévia durante a fase de desenvolvimento e evitando a propagação de falhas para o ambiente de produção.

A quantidade de testes, isoladamente, não garante a eficácia na detecção de falhas [Inozemtseva and Holmes 2014]. Portanto, para validar os próprios testes aplicou-se a técnica de testes de mutação (*mutation testing*) [Jia and Harman 2011], que insere falhas intencionais no código fonte para verificar quantas foram identificadas pelos testes.

Durante o início do desenvolvimento, uma especificação começou a ser iterativamente refinada até que atendesse a critérios suficientes para um MVP (*minimum viable product*) [Ries 2011]. Esse início utilizava a abordagem de uma arquitetura cliente-servidor, pois acreditava-se que as ferramentas para construção de interfaces *web* combinadas com bibliotecas de cientometria do ecossistema Python seriam essenciais para o avanço do sistema. Esse MVP tinha duas funções básicas, a busca unificada em múltiplas bases e a leitura interativa de PDFs salvos na ferramenta, permitindo ao usuário marcar trechos do texto e associar anotações.

Entretanto, observaram-se algumas limitações nesse MVP, seu objetivo era primariamente facilitar o processo de leitura e coleta desses dados, uma vez que sua análise bibliométrica já é uma tarefa bem cumprida por pacotes como bibliometrix [Aria and Cuccurullo 2017], portanto a forma mais eficaz de auxiliar a análise bibliométrica seria permitir exportação em um formato que essas ferramentas já existentes e maduras suportam. Adotando essa premissa, o sistema foi adaptado para permitir a exportação de dados salvos em CSL [D'Arcus et al. 2021] para o formato padrão de exportação utilizado pela Scopus, e bem interpretado pelo bibliometrix.

Com a utilização do ecossistema Python para cientometria deixando de se tornar uma prioridade, notou-se ser válida a migração da arquitetura cliente-servidor para uma arquitetura mais coesa e independente, adotando o formato padrão de aplicação desktop. As ferramentas avaliadas para construção da aplicação foram os frameworks Tauri e Electron, sendo a segunda opção a escolha consolidada para seguir o desenvolvimento, por motivos de conhecimentos prévios da equipe disponível.

A partir dessa etapa, as novas versões da aplicação foram desenvolvidas e refinadas seguindo princípios do *Extreme Programming* [Beck 2000] e do desenvolvimento ágil de software [Beck et al. 2001], com entrega contínua, testes unitários, estilo de código

padronizado, falhas rápidas e refatoração frequente. Eventualmente, o desenvolvimento de novas features era pausado para realizar uma auditoria avaliando tratamento de erros, segurança, desempenho, qualidade e cobertura de testes e qualidade de código, geralmente seguidas de correções cabíveis considerando os resultados encontrados. Feedbacks eram frequentemente coletados para direcionar os próximos passos da aplicação, avaliando o desempenho da ferramenta em cenários reais de projetos de pesquisa de iniciação científica.

Também foram realizadas integrações com modelos de inteligência artificial, especialmente LLMs (*Large Language Models*) [Brown et al. 2020] e modelos de *embedding* [Muennighoff et al. 2023] para realizar tarefas como preenchimento de lacunas nos metadados dos artigos de forma não destrutiva, gerar resumos do artigo que está sendo lido e uma investigação massiva.

O recurso da investigação massiva recebe uma lista de perguntas do usuário, e processa os artigos de forma individual realizando uma busca em um banco vetorial, com os artigos já processados pelo modelo de *embedding*, para encontrar os trechos do artigo mais relevantes para responder cada pergunta. Com os top-k trechos mais relevantes [Lewis et al. 2020], ele entrega os trechos úteis para um modelo de LLM sintetizar o conteúdo na forma de uma resposta para a pergunta feita, registrando de forma transparente o *score* de confiança da resposta e da relevância de cada trecho fornecido. A Figura 1 demonstra esse processo de forma visual.

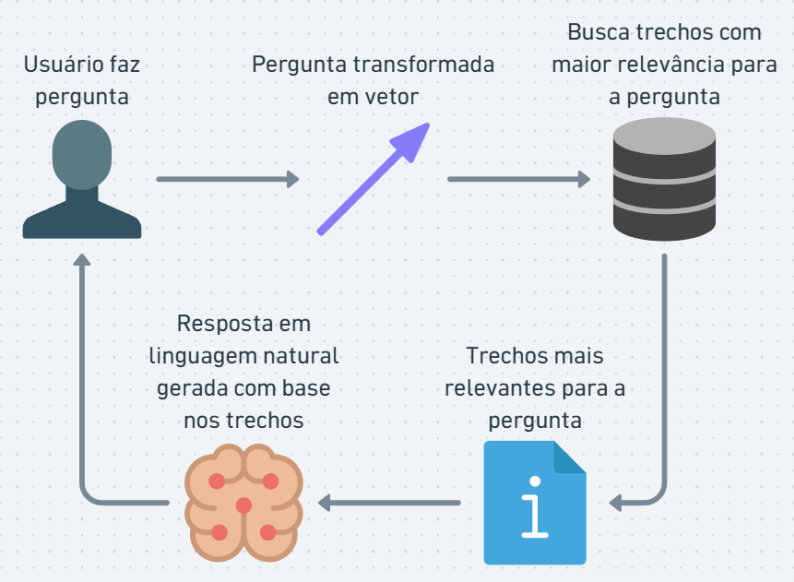


Figura 1. Fluxograma da investigação massiva.

3. Resultados e discussão

Para exemplificar o uso da aplicação, foi realizada uma pesquisa rápida sobre o tema "Previsão Climática de Precipitação", aplicando a investigação com RAG a 17 artigos selecionados através de duas buscas. A Figura 2 mostra a página inicial do projeto utilizado como exemplo, onde duas pesquisas foram realizadas, com 96 trabalhos resultantes.

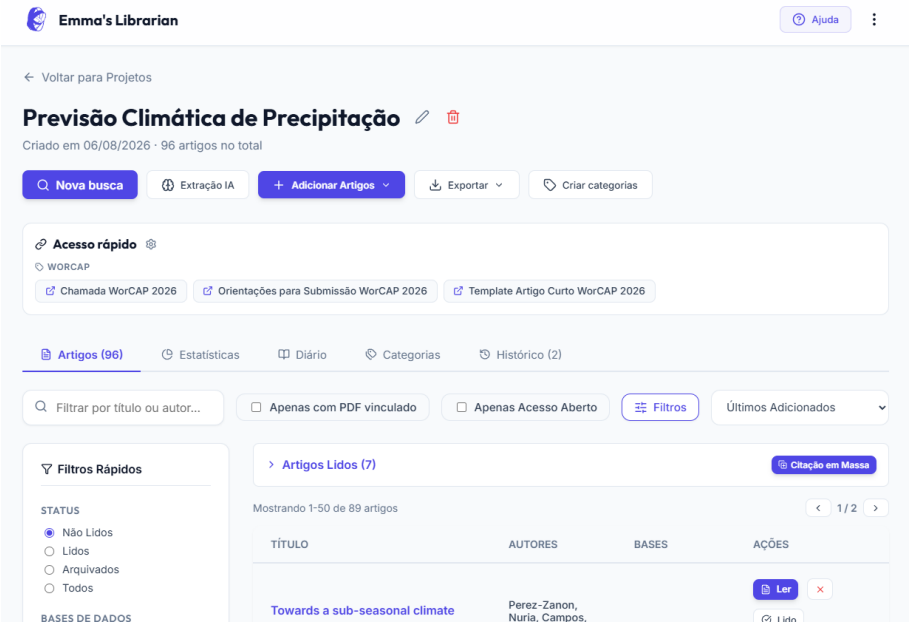


Figura 2. Tela de detalhes de um projeto de exemplo.

O Emma's Librarian se posiciona como uma ferramenta de apoio aos pesquisadores, buscando integrar-se ao seu processo de trabalho de forma a acelerar a pesquisa ao longo de múltiplos projetos, sem comprometer confiabilidade e transparência.

4. Conclusão

Este trabalho apresentou o desenvolvimento e a arquitetura do Emma's Librarian, uma aplicação concebida para auxiliar pesquisadores em variados estágios de experiência na condução e gestão de múltiplos projetos de pesquisa. Ao centralizar as buscas bibliográficas, a organização documental e a análise do estado da arte através de modelos de inteligência artificial, a ferramenta responde de forma direta à crescente fragmentação tecnológica enfrentada na rotina acadêmica.

O desenvolvimento de software assistido por inteligência artificial, sustentado por rigorosas práticas de Engenharia de Software como TDD e testes de mutação, assegura que a exploração de artigos seja feita sem sacrificar a confiabilidade, a transparência das fontes e a reprodutibilidade dos dados. Dessa forma, a ferramenta traz implicações significativas para a otimização de tempo do pesquisador, reduzindo a sobrecarga cognitiva gasta na transição entre múltiplas plataformas.

O sistema encontra-se em desenvolvimento contínuo, sendo que os próximos passos do desenvolvimento visam otimizar o balanceamento das consultas a bases bibliográficas, com o objetivo de adaptar as buscas estruturadas aos limites das APIs, e refinar os métodos de busca semântica, integrando continuamente novas funcionalidades em resposta ao feedback da comunidade científica.

Referências

Akita, F. M. (2026). Do zero à pós-produção em 1 semana - como usar ia em projetos de verdade — bastidores do the m.akita chronicles.

Disponível em: https://akitaonrails.com/2026/02/20/do-zero-a-pos-producao-em-1-semana-como-usar-ia-em-projetos-de-verdade-bastidores-do-the-m-akita-chronicles/.

Aria, M. and Cuccurullo, C. (2017). bibliometrix: An r-tool for comprehensive science mapping analysis. *Journal of Informetrics*, 11(4):959–975.

Beck, K. (2000). *Extreme Programming Explained: Embrace Change*. Addison-Wesley Professional, Reading, MA.

Beck, K. (2002). *Test-Driven Development: By Example*. Addison-Wesley Professional, Boston, MA.

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., and Thomas, D. (2001). Manifesto for agile software development. Disponível em: https://agilemanifesto.org/.

Bramer, W. M., de Jonge, G. B., Rethlefsen, M. L., Mast, F., and Kleijnen, J. (2018). A systematic approach to searching: an efficient and complete method to develop literature searches. *Journal of the Medical Library Association*, 106(4):531–541.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., et al. (2020). Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, volume 33, pages 1877–1901.

D'Arcus, B., Zelle, R. M., Bennett, Frank G., J., Wiernik, B. M., and Maier, D. (2021). Citation style language 1.0.2 specification. Technical report, Citation Style Language Project.

Inozemtseva, L. and Holmes, R. (2014). Coverage is not strongly correlated with test suite effectiveness. In *Proceedings of the 36th International Conference on Software Engineering*, ICSE '14', pages 435–445. ACM.

Jia, Y. and Harman, M. (2011). An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering*, 37(5):649–678.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., and Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pages 9459–9474. Curran Associates, Inc.

Muennighoff, N., Tazi, N., Magne, L., and Reimers, N. (2023). MTEB: Massive text embedding benchmark. In *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, pages 2014–2037. Association for Computational Linguistics.

Ries, E. (2011). *The Lean Startup: How Today's Entrepreneurs Use Continuous Innovation to Create Radically Successful Businesses*. Crown Business, New York, NY.

Yang, J., Jimenez, C. E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., and Press, O. (2024). Swe-agent: Agent-computer interfaces enable automated software engineering. In *Advances in Neural Information Processing Systems*, volume 37.